

MODULE 02

JavaScript Data: Working with Collections

Higher-order functions, safe transformations, and modern syntax for processing arrays and objects.

LEA LABS

Arrays Are Smart Collections

Arrays in JavaScript are more than just lists of data. They come equipped with built-in methods that make adding, removing, and finding items reliable and readable.

MANUAL MANAGEMENT

Manually tracking the length of an array and assigning items to specific index numbers is error-prone and hard to read.

BUILT-IN METHODS

Methods like `push()`, `pop()`, and `splice()` handle the mechanics for you.

MANAGING A LIST

```
const stack = ["HTML", "CSS"];  
// manual, fragile way  
stack[stack.length] = "JS";  
// readable, safe way  
stack.push("React");  
console.log(stack); // ["HTML", "CSS",  
"JS", "React"]  
const lastLearned = stack.pop();  
console.log(lastLearned); // "React"
```

Functions as Values

In JavaScript, functions are values you can store and pass — including as arguments to other functions.

THE ANALOGY

Passing data is like handing someone a cake. Passing a function is like handing them the **recipe**.

CALLBACKS

A function passed into another function is a **callback**. The receiver calls back to your logic when needed.

PASSING LOGIC

```
function greet(name) {  
  console.log(`Hello, ${name}!`);  
}  
  
function processUser(userName, action) {  
  action(userName);  
}  
  
processUser("Alice", greet);
```

Transforming Data with map()

The `map()` method creates a **new array** populated with the results of calling a provided function on every element in the calling array. It transforms data without mutating the original list.

ORIGINAL ARRAY

[1, 2, 3]



CALLBACK: (X) => X * 10



NEW ARRAY

[10, 20, 30]

FORMATTING PRICES

```
const prices = [45000, 35000, 18000];
```

```
const formatted = prices.map(p => `KES ${p}`);
```

```
console.log(formatted);
```

```
console.log(prices);
```

Selecting Data with `filter()`

When you need a subset of an array, use `filter()`. It tests each item and keeps only those that pass the condition.

THE TEST CALLBACK

You provide a callback function that returns `true` (keep it) or `false` (discard it) for each item.

A NEW ARRAY

Like `map()`, `filter()` does not change the original array. It returns a brand new array containing only the passing items.

FILTERING ACTIVE LEARNERS

```
const learners = [{name: "Alice", active: true}, {name: "Bob", active: false}, {name: "Charlie", active: true}];
const activeLearners = learners.filter(l => l.active === true);
console.log(activeLearners.length); // 2
```

Summarizing Data with `reduce()`

When you need to calculate a single result from an entire array—like a total price or a combined string—`reduce()` is the tool for the job.

THE ACCUMULATOR

The callback function receives an **accumulator** (the running total) and the **current item**. It returns the new accumulator value for the next step.

THE INITIAL VALUE

You must provide a starting value as the second argument to `reduce()`. For a sum, this is usually `0`.

CALCULATING A TOTAL

```
const cart = [45000, 35000, 18000];

// reduce(callback, initialValue)
const total = cart.reduce((acc, price)
=> {
  return acc + price;
}, 0);

console.log(total); // 98000
```

Chaining Array Methods

Because methods like `filter()` and `map()` return new arrays, you can call the next method directly on the result. This creates a clean, readable data pipeline.

1. RAW ARRAY



`.filter(isActive)`



`.map(getPrice)`



`.reduce(sum)`



FINAL TOTAL

THE PIPELINE IN ACTION

```
const courses =  
  [{name: "HTML", price: 100, active: true},  
   {name: "CSS", price: 150, active: false},  
   {name: "JS", price: 200, active: true}];
```

```
const activeRevenue = courses.filter(c =>  
  c.active).map(c => c.price).reduce((sum, price)  
=> sum+price, 0);
```

```
console.log(activeRevenue); // 300
```

Unpacking Data Quickly

Destructuring assignment is a special syntax that allows us to "unpack" arrays or objects into distinct variables, making our code cleaner and more readable.

OBJECT DESTRUCTURING

Extract properties by their exact key name. Order doesn't matter.

EXTRACTING BY KEY

```
const course = { title: "Applied AI",  
price: 35000 };
```

// Modern

```
const { title, price } = course;
```

ARRAY DESTRUCTURING

Extract items by their position in the array. Variable names are up to you.

EXTRACTING BY POSITION

```
const scores = [85, 92, 100];
```

// Modern

```
const [ first, second ] = scores;
```


The Spread Operator

The spread operator (`...`) expands arrays or objects into places where elements or arguments are expected.

SAFE COPYING

Spread creates a shallow copy of arrays/objects so the original isn't mutated by accident.

EASY MERGING

Combine arrays or merge object properties into a new structure with minimal syntax.

SPREADING ARRAYS

```
const groupA = ["Alice",  
"Bob"], groupB =  
["Charlie"], combined =  
[...groupA, "Zack",  
...groupB];  
console.log(combined);
```

SPREADING OBJECTS

```
const defaults = { theme:  
"light", role: "user" },  
userPrefs = { theme:  
"dark" }, settings = {  
...defaults, ...userPrefs  
};  
console.log(settings);
```

Predictable Pure Functions

When processing data, we want our functions to be predictable. A "pure" function follows two strict rules that make bugs much easier to track down.

1. DETERMINISTIC

Given the exact same input, a pure function will always return the exact same output. It doesn't rely on random numbers, the current time, or external variables.

2. NO SIDE EFFECTS

A pure function does not modify anything outside of itself. It doesn't change external variables, mutate the original array, or update the DOM.

✗ IMPURE
(UNPREDICTABLE)

```
let taxRate = 0.16;
```

```
function  
addTax(price) {  
  return price +  
  (price * taxRate);  
}
```

✓ PURE (PREDICTABLE)

```
function  
addTaxPure(price,  
rate) {  
  return price +  
  (price * rate);  
}
```

```
addTaxPure(100,  
0.16);
```

Lab: Data Processing Pipeline

It is time to put higher-order functions to work. You will receive a raw dataset of LEA Labs learners and build a pure data pipeline to extract key insights.

THE MISSION

- **Filter:** Remove any learners whose enrollment status is inactive or suspended.
- **Map:** Transform the remaining objects to extract only the learner's name and their final project score.
- **Reduce:** Calculate the average project score for the active cohort.

THE RULES

- **No Mutation:** You must not alter the original array of learner objects at any point.
- **Chain Methods:** Combine your array methods into a single, readable pipeline.
- **Pure Functions:** Ensure all callbacks provided to your methods are deterministic and have no side effects.